

Assembler language

Reserving and initializing storage

Introduction

- In this section, we'll learn:
 - How to define areas of storage with initial values
 - This is done with the Assembler instruction **DEFINE CONSTANT (DC)**
 - How to reserve an area of storage without any contents
 - Done using **DEFINE STORAGE (DS)**
 - How to use **literals** to define and represent data
 - Presented after **DC** and **DS**

Reserving storage with an initial value

- The Assembler instruction `DEFINE CONSTANT` (mnemonic **DC**) reserves an area of storage with an initial value
 - Note the area is not really a constant, as it can be changed during program execution
- We'll learn how to use `DC` for these data types:
 - **Binary** integers
 - **Characters**
 - **Hexadecimal** digits
 - **Bits**
- Other data types, such as **address** and **decimal**, we'll cover later

Assembler language

Reserving and initializing storage

Part 1: binary integers

Binary integers - brief recap

- We know that binary integers can be halfwords, fullwords, or doublewords
- Halfwords occupy 16 bits (2 bytes), on a halfword boundary (an even address)
- Fullwords occupy 32 bits (4 bytes), on a fullword boundary (a multiple-of-4 address)
- Doublewords occupy 64 bits (8 bytes), on a doubleword boundary (a multiple-of-8 address)

Defining binary integers

- To define a binary halfword, we use, for example:

DC

H'100'

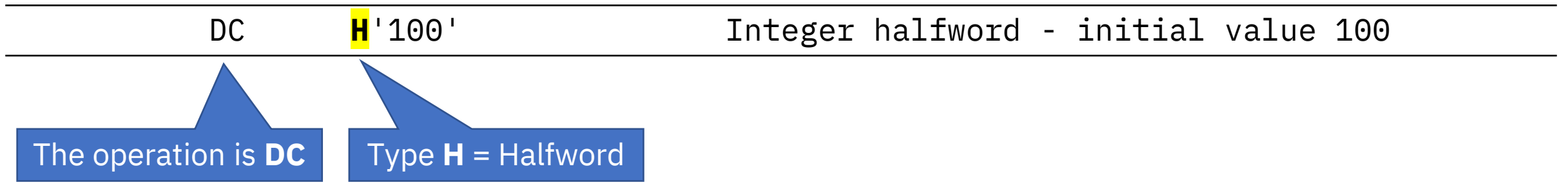
Integer halfword - initial value 100



The operation is **DC**

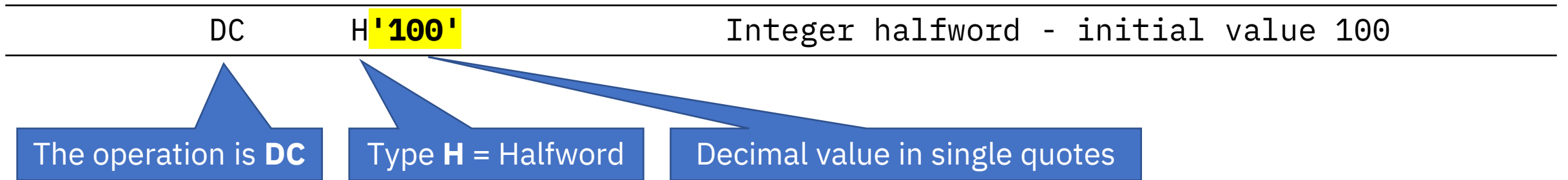
Defining binary integers

- To define a binary halfword, we use, for example:



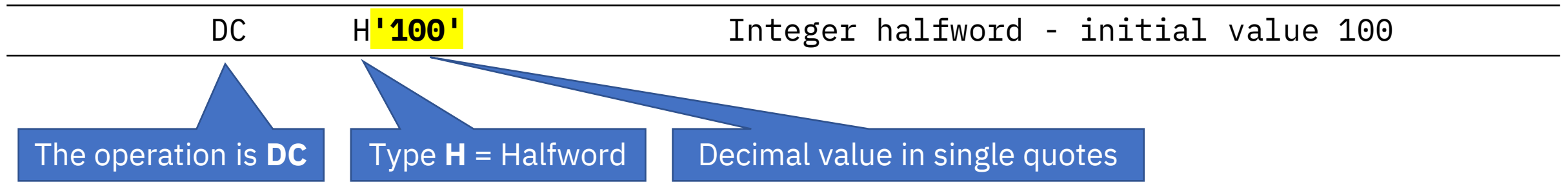
Defining binary integers

- To define a binary halfword, we use, for example:



Defining binary integers

- To define a binary halfword, we use, for example:



- This generates, *at a halfword boundary*:

X'0064' (X'64' is decimal 100)

Defining binary integers

- In most cases, the statement will have a label, so instructions can refer to the field:

Price	DC	H'100'	Unit price
--------------	----	--------	------------

- To add the halfword at `Price` to register 3:

AH	3,Price	Add Price to register 3
----	---------	-------------------------

(AH - ADD HALFWORD, adds the second operand halfword to the register, and places the result in the register)

Defining binary integers

- You can also define negative values:

Discount	DC	H'-5'
----------	----	-------

- This generates, at a halfword boundary:

X'FFFB' (decimal -5)

Defining binary integers - fullwords

- To define a fullword, the type is **F**. Examples:

Int1	DC	F ' 65535 '	Generates X'0000FFFF'
Int2	DC	F ' -1 '	Generates X'FFFFFFFF'
Max_F	DC	F ' 2147483647 '	Highest positive - X'7FFFFFFF'
Min_F	DC	F ' -2147483648 '	Lowest negative - X'80000000'
Min_2	DC	F ' -2 147 483 648 '	Also X'80000000' - but more readable!

- The values are placed at a *fullword boundary*

Defining binary integers - fullwords

- To define a fullword, the type is **F**. Examples:

Int1	DC	F ' 65535 '	Generates X'0000FFFF'
Int2	DC	F ' -1 '	Generates X'FFFFFFFF'
Max_F	DC	F ' 2147483647 '	Highest positive - X'7FFFFFFF'
Min_F	DC	F ' -2147483648 '	Lowest negative - X'80000000'
Min_2	DC	F' -2 147 483 648 '	Also X'80000000' - but more readable!

- The values are placed at a *fullword boundary*
- Note the last example: the Assemble ignores spaces, but they are useful for making the definitions easier to read

Defining binary integers - doublewords

- To define a doubleword, we also use type **F**, but with a **D** *type extension*:

DC	FD '0'	Generates X'0000000000000000'
DC	FD '1'	Generates X'0000000000000001'

Diagram illustrating the components of the **FD** type definition:

- Type F** (points to the **F** in **FD**)
- Type extension D** (points to the **D** in **FD**)

- Type extension **D** indicates a doubleword
- The values are generated at a *doubleword boundary*

Defining binary integers - a note about type D

- You may find that many programs use *type* **D** to define a doubleword with all zeroes.
- It works the same as **FD'0'**

DC	FD '0'	Generates X'0000000000000000'
DC	D '0'	Generates X'0000000000000000'

- So why not use **D** instead of **FD** to define doubleword integers then?
- Because **D** only works for zero! (Next slide explains...)

Defining binary integers - a note about type D

- Type D defines a **long floating-point number**, *not a binary doubleword!*
- Many programmers are shocked when they code

DC

D'1'

Expecting X'00000000000000001'

- ...and the Assembler generates this!

X'4110000000000000'

- (We won't cover floating point formats in this course; *just remember to use **FD**, not **D***, when defining doubleword binary integers)

Duplication factor

- It is possible to define a repeated field by using a *duplication factor*
- For example, this generates three identical halfwords:

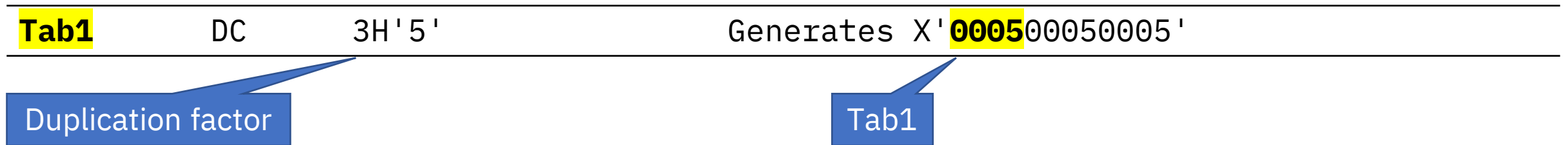
Tab1	DC	3 H'5'	Generates X'000500050005'
------	----	---------------	----------------------------------



Duplication factor

Duplication factor

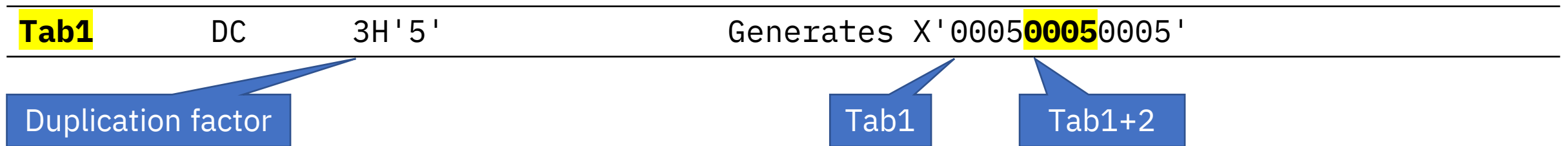
- It is possible to define a repeated field by using a *duplication factor*
- For example, this generates three identical halfwords:



- The label, Tab1, *refers to the first element, not the whole array*

Duplication factor

- It is possible to define a repeated field by using a *duplication factor*
- For example, this generates three identical halfwords:



- The label, **Tab1**, *refers to the first element, not the whole array*
 - To refer to the second element you could use, for example, **Tab1+2**

Duplication factor

- A duplication factor of zero does not generate any data, but forces alignment:

Duplication factor zero		Value can be omitted, as no data generated	
DC	0F	Generates nothing...	
DC	...	...but this is now fullword-aligned	

- Either of these is often used to ensure doubleword alignment:

DC	0FD	Force doubleword alignment
DC	0D	Force doubleword alignment

About alignment

- Let's look at how the Assembler aligns your fields
- For that, we'll use a simplified version of the listing the Assembler produces when it processes your program
- Given a sequence of statements like this...

DC	FD ' 1 '
DC	F ' 2 '
DC	H ' 3 '
DC	H ' 4 '

- ...the Assembler will produce a listing, shown next

About alignment

- This is a simplified version of an Assembler listing
 - Columns were omitted for readability

Location	Object Code	Source Statement	
000000	00000000000000000001	DC	FD '1 '
000008	000000002	DC	F '2 '
00000C	0003	DC	H '3 '
00000E	0004	DC	H '4 '

- See next slide for a description

About alignment

Location	Object Code	Source Statement	
000000	00000000000000000001	DC	FD '1'
000008	00000002	DC	F '2'
00000C	0003	DC	H '3'
00000E	0004	DC	H '4'

- **Location** is the position in your program where the Assembler places data or instructions

About alignment

Location	Object Code	Source Statement	
000000	00000000000000000001	DC	FD '1 '
000008	00000002	DC	F '2 '
00000C	0003	DC	H '3 '
00000E	0004	DC	H '4 '

- **Location** is the position in your program where the Assembler places data or instructions
- **Object code** is what the Assembler generates from your source code

About alignment

Location	Object Code	Source Statement	
000000	00000000000000000001	DC	FD '1 '
000008	000000002	DC	F '2 '
00000C	0003	DC	H '3 '
00000E	0004	DC	H '4 '

- **Location** is the position in your program where the Assembler places data or instructions
- **Object code** is what the Assembler generates from your source code
- **Source statement** is a copy of your code

About alignment

Location	Object Code	Source Statement	
000000	00000000000000000001	DC	FD '1 '
000008	000000002	DC	F '2 '
00000C	0003	DC	H '3 '
00000E	0004	DC	H '4 '

- **Location** is the position in your program where the Assembler places data or instructions
- **Object code** is what the Assembler generates from your source code
- **Source statement** is a copy of your code
- Note all the fields are correctly aligned

About alignment

- If we reorder the statements like this...

DC	H' 3'
DC	F' 2'
DC	H' 4'
DC	FD' 1'

About alignment

- ...the Assembler introduces gaps to ensure correct alignment:

Location	Object Code	Source Statement
000000	0003	DC H'3'
000002	0000	
000004	00000002	DC F '2'
000008	0004	DC H'4'
00000A	00000000000000	
000010	000000000000000001	DC FD '1'

About alignment

- ...the Assembler produces “gaps”, filled with X' 00 ', to ensure alignment:

Location	Object Code	Source Statement	
000000	0003	DC	H' 3 '
000002	0000		
000004	00000002	DC	F ' 2 '
000008	0004	DC	H' 4 '
00000A	00000000000000		
000010	00000000000000001	DC	FD' 1 '

- The two-byte gap at location 000002 ensures that the following fullword is at a multiple-of-4 address

About alignment

- ...the Assembler produces “gaps”, filled with X' 00 ', to ensure alignment:

Location	Object Code	Source Statement	
000000	0003	DC	H' 3 '
000002	0000		
000004	00000002	DC	F' 2 '
000008	0004	DC	H' 4 '
00000A	00000000000000		
000010	000000000000000001	DC	FD ' 1 '

- The two-byte gap at location 000002 ensures that the following fullword is at a multiple-of-4 address
- Same for the six bytes at 00000A, which ensure correct alignment for the doubleword

Field length

- We can specify a field length when using DC, using **L** (called **length modifier**) after the type
- It's unlikely that you will need to specify a length for binary integers, as they have an implicit length. We introduce it here for completeness
- Example: the following are equivalent and define a 1-byte binary integer:

Location	Object Code	Source Statement	
000000	64	DC	H L1 '100'
000001	64	DC	F L1 '100'
000002	64	DC	FD L1 '100'

Field length

- Here are a few more examples:

Location	Object Code	Source Statement	
000000	01	DC	FL1 '1'
000001	0001	DC	FL2 '1'
000003	000001	DC	FL3 '1'
000006	00000001	DC	FL4 '1'
00000A	0000000001	DC	FL5 '1'

- Note that, *when there is an explicit length, the Assembler does not enforce alignment*

Defining binary integers - summary

- To define binary integers, we use the **DC** instructions with types:
 - **H** for halfword
 - **F** for fullword
 - **FD** for doubleword (the **D** in **FD** is a *type extension*)
- We specify the value in decimal, in single quotes
 - We can specify negative values
 - Spaces in values are ignored, but can be used for readability

Defining binary integers - summary

- We can specify a *duplication factor* to define multiple, consecutive fields
 - A label, if present, refers to the first of the repeated fields
 - When zero, a duplication factor generates no data by ensures alignment
- The generated fields are aligned (unless specifying an explicit length)
- Explicit length is rarely used when defining binary integers

Define Constant - summary

- The format of the Define Constant statement is:

DC

dTxLn 'value'

- Operation: **DC**

Define Constant - summary

- The format of the Define Constant statement is:

DC	d TxLn 'value'
----	-----------------------

- Operation: **DC**
- Duplication factor (**d**): optional

Define Constant - summary

- The format of the Define Constant statement is:

DC	d	T	xLn 'value'
----	---	----------	-------------

- Operation: **DC**
- Duplication factor (**d**): optional
- Type (**T**): required

Define Constant - summary

- The format of the Define Constant statement is:

DC	d	T	x	L	n	'	v	a	l	u	e	'
----	---	---	---	---	---	---	---	---	---	---	---	---

- Operation: **DC**
- Duplication factor (**d**): optional
- Type (**T**): required
- Type extension (**x**): optional

Define Constant - summary

- The format of the Define Constant statement is:

DC	dTx	Ln	'value'
----	-----	-----------	---------

- Operation: **DC**
- Duplication factor (**d**): optional
- Type (**T**): required
- Type extension (**x**): optional
- Length modifier (**L**): optional

Define Constant - summary

- The format of the Define Constant statement is:

DC	dTxLn	'value'
----	-------	---------

- Operation: **DC**
- Duplication factor (**d**): optional
- Type (**T**): required
- Type extension (**x**): optional
- Length (**L**): optional
- Value: required (unless duplication factor is zero)

Reserving space

- `DEFINE STORAGE` (mnemonic **DS**) reserves space without an initial value
- Fields are aligned if necessary
- For example:

H1	DS	H	Reserve space for a halfword
F1	DS	F	Reserve space for a fullword
FD1	DS	FD	Reserve space for a doubleword

- The statements above produce the listing shown next

Reserving space

Location	Object Code	Source Statement			
000000		H1	DS	H	Reserve space for a halfword
000004		F1	DS	F	Reserve space for a fullword
000008		FD1	DS	FD	Reserve space for a doubleword

- Note that the Assembler places the fields with the correct alignment
 - The halfword at 000000 occupies 2 bytes
 - The next available byte after the halfword is at 000002, but the fullword starts at 000004
- Also note no data is shown in the “Object code” column
 - The reserved space will normally be all X' 00 '

Reserving space

- More examples:

H3	DS	3H	Reserve space for 3 halfwords
F1	DS	FL1	Reserve 1 byte
FD1	DS	F '10000'	Value is for documentation only

- **DS** accepts the same options as **DC**
 - Including value!
- **DS** ignores the value; it is for documentation only
 - This could lead to program errors if you meant to use **DC**!

Defining binary integers - the end

- This concludes the section about defining binary integers
- Next, we'll see how to define character strings